6 - Listas

Alejandro Veloz

aavelozb.github.io/py



Objetivos de la unidad

- **Almacenar y procesar** elementos en listas: recorrerlas, agregar, buscar y borrar elementos, y crear sublistas.
- Implementar **algoritmos básicos** sobre listas.
- Resolver problemas **alterando el orden** de una lista o **construyendo listas nuevas** a partir de otras.
- Resolver problemas con **listas de listas**, construyendo y recorriendo la estructura con el patrón de **ciclo anidado**.

El problema

Leer 5 notas y calcular el promedio: se puede con lo que ya sabemos.

```
suma = 0
i = 0
while i < 5:
    suma += float(input('Nota: '))
    i += 1
print(suma/5)
```

El problema

Leer 5 notas y calcular el promedio: se puede con lo que ya sabemos.

```
suma = 0
i = 0
while i < 5:
    suma += float(input('Nota: '))
    i += 1
print(suma/5)
```

Ahora además hay que mostrar **cuáles están sobre el promedio**.

Imposible: el promedio se conoce recién al final, y los datos ya se perdieron.

El problema

Una variable por nota:

```
n1 = float(input('Nota: '))
n2 = float(input('Nota: '))
n3 = float(input('Nota: '))
n4 = float(input('Nota: '))
n5 = float(input('Nota: '))
```

El problema

Una variable por nota:

```
n1 = float(input('Nota: '))
n2 = float(input('Nota: '))
n3 = float(input('Nota: '))
n4 = float(input('Nota: '))
n5 = float(input('Nota: '))
```

No escala: ¿y si son 500?

No se puede recorrer: no hay forma de escribir un ciclo sobre `n1`, `n2`, …

El problema

Una variable por nota:

```
n1 = float(input('Nota: '))
n2 = float(input('Nota: '))
n3 = float(input('Nota: '))
n4 = float(input('Nota: '))
n5 = float(input('Nota: '))
```

No escala: ¿y si son 500?

No se puede recorrer: no hay forma de escribir un ciclo sobre `n1`, `n2`, …

Necesitamos **una** variable que guarde **muchos** valores.

Listas: lo básico

¿Qué es una lista?

Una **colección ordenada** de elementos, de cualquier tipo. Se escribe entre **corchetes**, con los elementos separados por comas.

```
notas   = [55, 70, 48, 91]
colores = ['azul', 'rojo', 'verde']
mezcla  = [1, 'hola', 3.5, True]
vacia   = []
```

¿Qué es una lista?

Una **colección ordenada** de elementos, de cualquier tipo. Se escribe entre **corchetes**, con los elementos separados por comas.

```
notas   = [55, 70, 48, 91]
colores = ['azul', 'rojo', 'verde']
mezcla  = [1, 'hola', 3.5, True]
vacia   = []
```

- Los elementos **no** tienen que ser del mismo tipo.
- Una lista puede contener **otras listas**.
- La lista **vacía** se escribe `[]` y tiene largo `0`.

Índices y rebanadas

Igual que en los strings: `[ ]` para un elemento, `[a:b]` para una sublista.

```
colores = ['azul', 'rojo', 'verde', 'amarillo']
print(colores[0])      # 'azul'
print(colores[-1])     # 'amarillo'
print(colores[1:3])    # ['rojo', 'verde']
print(len(colores))    # 4
```

Índices y rebanadas

Igual que en los strings: `[ ]` para un elemento, `[a:b]` para una sublista.

```
colores = ['azul', 'rojo', 'verde', 'amarillo']
print(colores[0])      # 'azul'
print(colores[-1])     # 'amarillo'
print(colores[1:3])    # ['rojo', 'verde']
print(len(colores))    # 4
```

- El primer índice es `0`; el último válido es `len(l) - 1`.
- Los índices negativos cuentan desde la derecha.
- Un índice inválido es un `IndexError`; una rebanada inválida, en cambio, no falla.

Mutabilidad

Aquí está **la gran diferencia** con los strings: una lista **sí se puede modificar**.

```
colores = ['azul', 'rojo', 'verde']
colores[0] = 'fucsia'
print(colores)     # ['fucsia', 'rojo', 'verde']
```

Mutabilidad

Aquí está **la gran diferencia** con los strings: una lista **sí se puede modificar**.

```
colores = ['azul', 'rojo', 'verde']
colores[0] = 'fucsia'
print(colores)     # ['fucsia', 'rojo', 'verde']
```

Compare con lo que ocurría con un string:

```
nombre = 'Bodoque'
nombre[0] = 'B'    # TypeError
```

Mutabilidad

Aquí está **la gran diferencia** con los strings: una lista **sí se puede modificar**.

```
colores = ['azul', 'rojo', 'verde']
colores[0] = 'fucsia'
print(colores)     # ['fucsia', 'rojo', 'verde']
```

Compare con lo que ocurría con un string:

```
nombre = 'Bodoque'
nombre[0] = 'B'    # TypeError
```

Las listas se modifican **en el lugar**: no hay que construir una nueva.

Listas y strings

	string	lista
contiene	caracteres	elementos de cualquier tipo
índices `[i]`, rebanadas `[a:b]`	sí	sí
`len`, `in`, `+`, `*`	sí	sí
recorrido con `for`	sí	sí
¿se puede **modificar**?	**no**	**sí**

Listas y strings

	string	lista
contiene	caracteres	elementos de cualquier tipo
índices `[i]`, rebanadas `[a:b]`	sí	sí
`len`, `in`, `+`, `*`	sí	sí
recorrido con `for`	sí	sí
¿se puede **modificar**?	**no**	**sí**

Los strings son **inmutables**; las listas, **mutables**. Esa única diferencia cambia la forma de resolver muchos problemas.

Listas y strings

	string	lista
contiene	caracteres	elementos de cualquier tipo
índices `[i]`, rebanadas `[a:b]`	sí	sí
`len`, `in`, `+`, `*`	sí	sí
recorrido con `for`	sí	sí
¿se puede **modificar**?	**no**	**sí**

Los strings son **inmutables**; las listas, **mutables**. Esa única diferencia cambia la forma de resolver muchos problemas.

Lo demás que aprendió sobre strings sirve igual aquí.

Operadores

+ concatena dos listas y crea una nueva.

```
a = [1, 2, 3]
b = a + [4, 5]
# [1, 2, 3, 4, 5]
```

*** repite** una lista.

```
ceros = [0] * 5
# [0, 0, 0, 0, 0]
```

Operadores

`+` **concatena** dos listas y crea una nueva.

```
a = [1, 2, 3]
b = a + [4, 5]
# [1, 2, 3, 4, 5]
```

`*` **repite** una lista.

```
ceros = [0] * 5
# [0, 0, 0, 0, 0]
```

`in` pregunta si un **elemento** está en la lista.

```
a = [1, 2, 3]
print(2 in a)          # True
print(5 not in a)      # True
```

Ojo: busca el elemento completo, no una parte de él.

Operadores

`+` **concatena** dos listas y crea una nueva.

```
a = [1, 2, 3]
b = a + [4, 5]
# [1, 2, 3, 4, 5]
```

`*` **repite** una lista.

```
ceros = [0] * 5
# [0, 0, 0, 0, 0]
```

`in` pregunta si un **elemento** está en la lista.

```
a = [1, 2, 3]
print(2 in a)          # True
print(5 not in a)      # True
```

Ojo: busca el elemento completo, no una parte de él.

`[0] * n` es la forma habitual de crear una lista de `n` casilleros.

Funciones sobre listas

```
notas = [55, 70, 48, 91]
print(len(notas))        # 4   cantidad de elementos
print(sum(notas))        # 264 suma (sólo números)
print(min(notas))        # 48
print(max(notas))        # 91
print(sum(notas)/len(notas))   # 66.0  el promedio
```

Funciones sobre listas

```
notas = [55, 70, 48, 91]
print(len(notas))        # 4   cantidad de elementos
print(sum(notas))        # 264 suma (sólo números)
print(min(notas))        # 48
print(max(notas))        # 91
print(sum(notas)/len(notas))   # 66.0  el promedio
```

`sum` sólo funciona si todos los elementos son números. `min` y `max` también sirven con strings, usando el orden lexicográfico.

Eliminar con `del`

`del l[i]` elimina el elemento que está en la posición `i`.

```
a = [1, 2, 3, 4, 2]
del a[0]
print(a)        # [2, 3, 4, 2]
del a[-1]
print(a)        # [2, 3, 4]
```

Eliminar con `del`

`del l[i]` elimina el elemento que está en la posición `i`.

```
a = [1, 2, 3, 4, 2]
del a[0]
print(a)        # [2, 3, 4, 2]
del a[-1]
print(a)        # [2, 3, 4]
```

- La lista **se achica**: los elementos que venían después corren una posición.
- `del` recibe una **posición**, no un valor. Para eliminar por valor está `remove`.

Métodos

Los métodos de las listas

Método	Qué hace
`l.append(x)`	agrega `x` **al final**
`l.insert(i, x)`	inserta `x` en la posición `i`
`l.remove(x)`	elimina la **primera** aparición de `x`
`l.index(x)`	posición de la **primera** aparición de `x`
`l.count(x)`	cuántas veces aparece `x`
`l.sort()`	ordena de menor a mayor
`l.reverse()`	invierte el orden

Los métodos de las listas

Método	Qué hace
`l.append(x)`	agrega `x` **al final**
`l.insert(i, x)`	inserta `x` en la posición `i`
`l.remove(x)`	elimina la **primera** aparición de `x`
`l.index(x)`	posición de la **primera** aparición de `x`
`l.count(x)`	cuántas veces aparece `x`
`l.sort()`	ordena de menor a mayor
`l.reverse()`	invierte el orden

Todos **modifican** la lista; ninguno retorna una lista nueva.

append: el método más usado

Agrega un elemento al final. Es la forma de **construir** una lista.

```
nros = []                  # la lista parte vacía
nros.append(10)            # [10]
nros.append(20)            # [10, 20]
nros.append(30)            # [10, 20, 30]
```

append: el método más usado

Agrega un elemento al final. Es la forma de **construir** una lista.

```
nros = []                  # la lista parte vacía
nros.append(10)            # [10]
nros.append(20)            # [10, 20]
nros.append(30)            # [10, 20, 30]
```

Es el equivalente del acumulador que ya conoce:

suma	`suma = 0`	`suma += x`
concatenación	`acum = ''`	`acum += c`
lista	`lista = []`	`lista.append(x)`

insert y remove

```
a = [1, 2, 3, 4]
a.insert(1, 10)
print(a)   # [1, 10, 2, 3, 4]
```

`insert(i, x)` deja `x` en la posición `i` y corre el resto a la derecha.

`insert` y `remove`

```
a = [1, 2, 3, 4]
a.insert(1, 10)
print(a)   # [1, 10, 2, 3, 4]
```

`insert(i, x)` deja `x` en la posición `i` y corre el resto a la derecha.

```
a = [1, 2, 3, 5, 3]
a.remove(3)
print(a)   # [1, 2, 5, 3]
```

`remove(x)` elimina **sólo la primera** aparición de `x`.

insert y remove

```
a = [1, 2, 3, 4]
a.insert(1, 10)
print(a)   # [1, 10, 2, 3, 4]
```

`insert(i, x)` deja `x` en la posición `i` y corre el resto a la derecha.

```
a = [1, 2, 3, 5, 3]
a.remove(3)
print(a)   # [1, 2, 5, 3]
```

`remove(x)` elimina **sólo la primera** aparición de `x`.

Si el elemento no está, `remove` produce un error. Conviene preguntar antes:

```
if 7 in a:
    a.remove(7)
```

index y count

```
a = [1, 2, 3, 4, 3]
print(a.count(3))     # 2
print(a.index(3))     # 2   (la primera aparición)
print(a.index(214))   # ValueError: 214 is not in list
```

index y count

```
a = [1, 2, 3, 4, 3]
print(a.count(3))     # 2
print(a.index(3))     # 2   (la primera aparición)
print(a.index(214))   # ValueError: 214 is not in list
```

Mismo cuidado que con `remove`: el elemento debe existir.

```
if 214 in a:
    print(a.index(214))
else:
    print('el elemento no está')
```

sort y reverse

```
a = [4, 1, 10, 6]
a.sort()
print(a)   # [1, 4, 6, 10]
```

```
b = ['juan', 'ana', 'luis']
b.sort()
# ['ana', 'juan', 'luis']
```

sort y reverse

```
a = [4, 1, 10, 6]
a.sort()
print(a)   # [1, 4, 6, 10]
```

```
b = ['juan', 'ana', 'luis']
b.sort()
# ['ana', 'juan', 'luis']
```

```
a = [4, 1, 10, 6]
a.reverse()
print(a)   # [6, 10, 1, 4]
```

`reverse` **no ordena**: sólo da vuelta la lista tal como está.

sort y reverse

```
a = [4, 1, 10, 6]
a.sort()
print(a)   # [1, 4, 6, 10]
```

```
b = ['juan', 'ana', 'luis']
b.sort()
# ['ana', 'juan', 'luis']
```

```
a = [4, 1, 10, 6]
a.reverse()
print(a)   # [6, 10, 1, 4]
```

`reverse` **no ordena**: sólo da vuelta la lista tal como está.

Para ordenar de **mayor a menor**, los dos en ese orden:

```
a.sort()
a.reverse()
```

Los métodos no retornan la lista

Este es el error más frecuente de la unidad:

```
a = [4, 1, 10, 6]
a = a.sort()
print(a)        # None
```

Los métodos no retornan la lista

Este es el error más frecuente de la unidad:

```
a = [4, 1, 10, 6]
a = a.sort()
print(a)        # None
```

`sort` ordena la lista y retorna `None`. Lo correcto es:

```
a.sort()        # sin asignar
print(a)        # [1, 4, 6, 10]
```

Los métodos no retornan la lista

Este es el error más frecuente de la unidad:

```
a = [4, 1, 10, 6]
a = a.sort()
print(a)        # None
```

`sort` ordena la lista y retorna `None`. Lo correcto es:

```
a.sort()        # sin asignar
print(a)        # [1, 4, 6, 10]
```

Por lo mismo, `a.sort().reverse()` no funciona: hay que escribir las dos instrucciones por separado.

Funciones y métodos

Funciones: `nombre(lista)`

```
len(a)
sum(a)
min(a)
max(a)
```

Retornan un valor y no tocan la lista.

Métodos: `lista.nombre()`

```
a.append(x)
a.sort()
a.remove(x)
a.index(x)
```

Modifican la lista. Sólo `index` y `count` retornan algo útil.

Construir y recorrer

Construir una lista

El patrón acumulador de siempre, ahora con append:

```
n = int(input('Cantidad de notas: '))
notas = []                          # antes del ciclo
i = 0
while i < n:
    notas.append(float(input('Nota: ')))
    i += 1
print(notas)
```

Construir una lista

El patrón acumulador de siempre, ahora con `append`:

```
n = int(input('Cantidad de notas: '))
notas = []                          # antes del ciclo
i = 0
while i < n:
    notas.append(float(input('Nota: ')))
    i += 1
print(notas)
```

La lista vacía se crea **antes** del ciclo. Si queda dentro, se pierde todo en cada vuelta.

Construir hasta un centinela

Ahora no hace falta saber cuántos datos son:

```
notas = []
dato = input('Nota (fin para terminar): ')
while dato != 'fin':
    notas.append(float(dato))
    dato = input('Nota (fin para terminar): ')
print('Promedio:', sum(notas)/len(notas))
```

Construir hasta un centinela

Ahora no hace falta saber cuántos datos son:

```
notas = []
dato = input('Nota (fin para terminar): ')
while dato != 'fin':
    notas.append(float(dato))
    dato = input('Nota (fin para terminar): ')
print('Promedio:', sum(notas)/len(notas))
```

Y ahora sí podemos volver sobre los datos: ya están todos guardados.

Recorrido con `for`

Igual que con los strings, `for` toma los elementos uno por uno:

```
ramos = ['Progra', 'Mate', 'Fisica']
for ramo in ramos:
    print('Tengo clases de', ramo)
```

Recorrido con `for`

Igual que con los strings, `for` toma los elementos uno por uno:

```
ramos = ['Progra', 'Mate', 'Fisica']
for ramo in ramos:
    print('Tengo clases de', ramo)
```

```
Tengo clases de Progra
Tengo clases de Mate
Tengo clases de Fisica
```

Recorrido con `for`

Igual que con los strings, `for` toma los elementos uno por uno:

```
ramos = ['Progra', 'Mate', 'Fisica']
for ramo in ramos:
    print('Tengo clases de', ramo)
```

```
Tengo clases de Progra
Tengo clases de Mate
Tengo clases de Fisica
```

La variable del `for` toma el **valor** de cada elemento, no su posición.

La función `range`

Genera una secuencia de números enteros. Se usa casi siempre junto a `for`.

```
for i in range(5):
    print(i)           # 0 1 2 3 4
```

La función `range`

Genera una secuencia de números enteros. Se usa casi siempre junto a `for`.

```
for i in range(5):
    print(i)           # 0 1 2 3 4
```

```
range(fin)
range(inicio, fin)
range(inicio, fin, salto)
```

```
list(range(5))
# [0, 1, 2, 3, 4]
list(range(2, 6))
# [2, 3, 4, 5]
list(range(2, 10, 3))
# [2, 5, 8]
list(range(10, 0, -2))
# [10, 8, 6, 4, 2]
```

La función `range`

Genera una secuencia de números enteros. Se usa casi siempre junto a `for`.

```
for i in range(5):
    print(i)           # 0 1 2 3 4
```

```
range(fin)
range(inicio, fin)
range(inicio, fin, salto)
```

```
list(range(5))
# [0, 1, 2, 3, 4]
list(range(2, 6))
# [2, 3, 4, 5]
list(range(2, 10, 3))
# [2, 5, 8]
list(range(10, 0, -2))
# [10, 8, 6, 4, 2]
```

El `fin` nunca se incluye, igual que en las rebanadas.

Recorrido por índice

Cuando necesitamos la **posición**, recorremos los índices válidos:

```
notas = [55, 70, 48, 91]
for i in range(len(notas)):
    print('Nota', i+1, ':', notas[i])
```

Recorrido por índice

Cuando necesitamos la **posición**, recorremos los índices válidos:

```
notas = [55, 70, 48, 91]
for i in range(len(notas)):
    print('Nota', i+1, ':', notas[i])
```

Es la traducción directa de lo que hacíamos con `while`:

```
i = 0
while i < len(notas):
    print('Nota', i+1, ':', notas[i])
    i += 1
```

¿Por elemento o por índice?

Por elemento

```
for n in notas:
    suma += n
```

- Más corto y más claro.
- Cuando sólo importa el **valor**.

Por índice

```
for i in range(len(notas)):
    notas[i] = notas[i] + 5
```

- Cuando hay que **modificar** la lista.
- Cuando hay que mirar **dos listas a la vez**.
- Cuando importa la **posición**.

¿Por elemento o por índice?

Por elemento

```
for n in notas:
    suma += n
```

- Más corto y más claro.
- Cuando sólo importa el **valor**.

Por índice

```
for i in range(len(notas)):
    notas[i] = notas[i] + 5
```

- Cuando hay que **modificar** la lista.
- Cuando hay que mirar **dos listas a la vez**.
- Cuando importa la **posición**.

Modificar `n` dentro del primer ciclo no cambia la lista: `n` es una copia del valor.

Dos cuidados con el `for`

(1) No cambie la variable del ciclo:

```
for i in range(len(a)):
    i += 2          # no sirve de nada: el for la reasigna
```

Dos cuidados con el `for`

(1) No cambie la variable del ciclo:

```
for i in range(len(a)):
    i += 2          # no sirve de nada: el for la reasigna
```

(2) No agregue ni elimine elementos mientras recorre:

```
for x in a:
    if x < 0:
        a.remove(x)     # se saltará elementos
```

Dos cuidados con el `for`

(1) No cambie la variable del ciclo:

```
for i in range(len(a)):
    i += 2          # no sirve de nada: el for la reasigna
```

(2) No agregue ni elimine elementos mientras recorre:

```
for x in a:
    if x < 0:
        a.remove(x)     # se saltará elementos
```

Para filtrar, construya una lista **nueva**.

Patrones con listas

Los patrones

#	Patrón	Idea
1	Construir	`lista = []` y `append` dentro del ciclo
2	Contar	contar los elementos que cumplen algo
3	Filtrar	construir una lista nueva con algunos
4	Mayor / menor	recorrer comparando, o usar `max` / `min`
5	Buscar	`in`, `index`, o recorrer con bandera
6	Listas paralelas	mismo índice en dos listas
7	Sin repetidos	agregar sólo si no está
8	Ciclo anidado	recorrer una lista de listas

2. Contar

Cuántos números pares hay en la lista:

```
nros = [4, 7, 2, 9, 10, 3]
pares = 0
for n in nros:
    if n % 2 == 0:
        pares += 1
print(pares, 'pares')
```

2. Contar

Cuántos números pares hay en la lista:

```
nros = [4, 7, 2, 9, 10, 3]
pares = 0
for n in nros:
    if n % 2 == 0:
        pares += 1
print(pares, 'pares')
```

Si lo que se cuenta es un valor exacto, el método `count` lo hace solo:

```
print(nros.count(7))      # 1
```

3. Filtrar

De una lista de números, construir una con los pares y otra con los impares:

```
nros = [4, 7, 2, 9, 10, 3]
pares = []
impares = []
for n in nros:
    if n % 2 == 0:
        pares.append(n)
    else:
        impares.append(n)
print(pares)      # [4, 2, 10]
print(impares)    # [7, 9, 3]
```

3. Filtrar

De una lista de números, construir una con los pares y otra con los impares:

```
nros = [4, 7, 2, 9, 10, 3]
pares = []
impares = []
for n in nros:
    if n % 2 == 0:
        pares.append(n)
    else:
        impares.append(n)
print(pares)      # [4, 2, 10]
print(impares)    # [7, 9, 3]
```

La lista original no se toca: se construyen listas nuevas.

4. Mayor / menor

A mano, con el patrón de siempre:

```
mayor = nros[0]
for n in nros:
    if n > mayor:
        mayor = n
```

4. Mayor / menor

A mano, con el patrón de siempre:

```
mayor = nros[0]
for n in nros:
    if n > mayor:
        mayor = n
```

Con la función:

```
mayor = max(nros)
```

4. Mayor / menor

A mano, con el patrón de siempre:

```
mayor = nros[0]
for n in nros:
    if n > mayor:
        mayor = n
```

Con la función:

```
mayor = max(nros)
```

Pero muchas veces no basta con el valor: si hay que saber **quién** tiene la nota máxima, el ciclo a mano es el que sirve.

```
mejor = 0
for i in range(len(notas)):
    if notas[i] > notas[mejor]:
        mejor = i
print(nombres[mejor], notas[mejor])
```

5. Buscar

Buscar la posición de un elemento, sin usar `index`:

```
nros = [4, 7, 2, 9]
buscado = 9
pos = -1
i = 0
while i < len(nros) and pos == -1:
    if nros[i] == buscado:
        pos = i
    i += 1
print(pos)       # 3
```

5. Buscar

Buscar la posición de un elemento, sin usar `index`:

```
nros = [4, 7, 2, 9]
buscado = 9
pos = -1
i = 0
while i < len(nros) and pos == -1:
    if nros[i] == buscado:
        pos = i
    i += 1
print(pos)       # 3
```

`pos = -1` hace de bandera: si al final sigue en `-1`, el elemento no estaba.

6. Listas paralelas

Dos listas del **mismo largo**, donde la posición `i` de una corresponde a la posición `i` de la otra.

```
ramos = ['Progra', 'Mate', 'Fisica']
horas = ['8:00',   '10:00', '12:00']

for i in range(len(ramos)):
    print('Tengo', ramos[i], 'a las', horas[i])
```

6. Listas paralelas

Dos listas del **mismo largo**, donde la posición `i` de una corresponde a la posición `i` de la otra.

```
ramos = ['Progra', 'Mate', 'Fisica']
horas = ['8:00',   '10:00', '12:00']

for i in range(len(ramos)):
    print('Tengo', ramos[i], 'a las', horas[i])
```

Aquí el `for` por elemento no sirve: necesitamos el índice para entrar a las dos listas a la vez.

7. Sin repetidos

Construir una lista nueva, en el mismo orden, pero sin elementos repetidos:

```
nros = [1, 2, 2, 3, 1, 4]
sin_rep = []
for n in nros:
    if n not in sin_rep:
        sin_rep.append(n)
print(sin_rep)       # [1, 2, 3, 4]
```

7. Sin repetidos

Construir una lista nueva, en el mismo orden, pero sin elementos repetidos:

```
nros = [1, 2, 2, 3, 1, 4]
sin_rep = []
for n in nros:
    if n not in sin_rep:
        sin_rep.append(n)
print(sin_rep)       # [1, 2, 3, 4]
```

La condición mira la lista que se está construyendo, no la original.

Alias y copia

Dos nombres, una sola lista

```
a = [1, 2, 3]
b = a            # b NO es una copia
b[0] = 99
print(a)         # [99, 2, 3]   <- cambió a
```

Dos nombres, una sola lista

```
a = [1, 2, 3]
b = a            # b NO es una copia
b[0] = 99
print(a)         # [99, 2, 3]   <- cambió a
```

a y b son **dos nombres para la misma lista**. Se les llama *alias*.

Dos nombres, una sola lista

```
a = [1, 2, 3]
b = a            # b NO es una copia
b[0] = 99
print(a)         # [99, 2, 3]   <- cambió a
```

a y b son **dos nombres para la misma lista**. Se les llama *alias*.

Esto no pasaba con números ni con strings, porque esos son inmutables: no hay forma de cambiarlos desde el otro nombre.

Copiar de verdad

Para obtener una lista **distinta** con los mismos elementos:

```
a = [1, 2, 3]
b = a[:]          # una rebanada completa es una copia
b[0] = 99
print(a)          # [1, 2, 3]   <- intacta
print(b)          # [99, 2, 3]
```

Copiar de verdad

Para obtener una lista **distinta** con los mismos elementos:

```
a = [1, 2, 3]
b = a[:]          # una rebanada completa es una copia
b[0] = 99
print(a)          # [1, 2, 3]   <- intacta
print(b)          # [99, 2, 3]
```

También sirve `list(a)` o `a + []`.

Copiar de verdad

Para obtener una lista **distinta** con los mismos elementos:

```
a = [1, 2, 3]
b = a[:]          # una rebanada completa es una copia
b[0] = 99
print(a)          # [1, 2, 3]   <- intacta
print(b)          # [99, 2, 3]
```

También sirve `list(a)` o `a + []`.

Regla práctica: si va a modificar una lista que recibió, copie primero.

Listas y funciones

Como la lista es mutable, una función **puede** modificar la lista que recibe:

```
def agregar_cero(lista):
    lista.append(0)        # modifica la lista original

a = [1, 2]
agregar_cero(a)
print(a)        # [1, 2, 0]
```

Listas y funciones

Como la lista es mutable, una función **puede** modificar la lista que recibe:

```
def agregar_cero(lista):
    lista.append(0)        # modifica la lista original

a = [1, 2]
agregar_cero(a)
print(a)        # [1, 2, 0]
```

En este curso no lo hacemos. Las funciones **retornan** una lista nueva:

```
def con_cero(lista):
    return lista + [0]
```

Listas de listas

Listas dentro de listas

Un elemento de una lista puede ser, a su vez, otra lista.

```
alumnos = [
    ['Alberto Gonzalez', 40, 30, 70],
    ['Francisca Almonacid', 100, 40],
    ['Pedro Reyes', 30, 50]
]
print(len(alumnos))      # 3  <- tres elementos
print(alumnos[1])        # ['Francisca Almonacid', 100, 40]
```

Listas dentro de listas

Un elemento de una lista puede ser, a su vez, otra lista.

```
alumnos = [
    ['Alberto Gonzalez', 40, 30, 70],
    ['Francisca Almonacid', 100, 40],
    ['Pedro Reyes', 30, 50]
]
print(len(alumnos))      # 3  <- tres elementos
print(alumnos[1])        # ['Francisca Almonacid', 100, 40]
```

Es la forma de guardar **registros**: cada elemento agrupa varios datos de una misma cosa.

Notación de doble índice

`l[i][j]`: primero se elige la lista interior, después el elemento dentro de ella.

```
alumnos[1]        # ['Francisca Almonacid', 100, 40]
alumnos[1][0]     # 'Francisca Almonacid'
alumnos[1][2]     # 40
alumnos[1][-1]    # 40
len(alumnos[1])   # 3
```

Notación de doble índice

`l[i][j]`: primero se elige la lista interior, después el elemento dentro de ella.

```
alumnos[1]        # ['Francisca Almonacid', 100, 40]
alumnos[1][0]     # 'Francisca Almonacid'
alumnos[1][2]     # 40
alumnos[1][-1]    # 40
len(alumnos[1])   # 3
```

Se lee de izquierda a derecha: `alumnos[1][0]` es *el elemento 0, del alumno 1*.

Matrices

Cuando todas las listas interiores tienen el mismo largo, tenemos una **tabla**:

```
años = [2018, 2017, 2016]
temps = [
    [25, 25, 22, 19, 16, 12, 11, 13, 15, 18, 23, 23],
    [27, 25, 22, 18, 16, 11, 12, 12, 14, 17, 19, 28],
    [24, 25, 22, 16, 13, 12, 11, 12, 13, 18, 22, 23]
]
print(temps[1][3])      # 18  -> año 2017, mes abril
```

Matrices

Cuando todas las listas interiores tienen el mismo largo, tenemos una **tabla**:

```
años = [2018, 2017, 2016]
temps = [
    [25, 25, 22, 19, 16, 12, 11, 13, 15, 18, 23, 23],
    [27, 25, 22, 18, 16, 11, 12, 12, 14, 17, 19, 28],
    [24, 25, 22, 16, 13, 12, 11, 12, 13, 18, 22, 23]
]
print(temps[1][3])      # 18  -> año 2017, mes abril
```

`temps[f][c]`: `f` elige la **fila** (el año) y `c` la **columna** (el mes).

Recorrer una lista de listas

Un ciclo para las filas y, dentro, otro para las columnas: el **ciclo anidado** de siempre.

```
suma = 0
for fila in temps:
    for t in fila:
        suma += t
print(suma / (len(temps) * len(temps[0])))
```

Recorrer una lista de listas

Un ciclo para las filas y, dentro, otro para las columnas: el **ciclo anidado** de siempre.

```
suma = 0
for fila in temps:
    for t in fila:
        suma += t
print(suma / (len(temps) * len(temps[0])))
```

Con índices, cuando hacen falta:

```
for f in range(len(temps)):
    for c in range(len(temps[f])):
        suma += temps[f][c]
```

Recorrer una lista de listas

El promedio de 2016 (la fila 2):

```
print(sum(temps[2]) / len(temps[2]))
```

Recorrer una lista de listas

El promedio de 2016 (la fila 2):

```
print(sum(temps[2]) / len(temps[2]))
```

El promedio de abril (la columna 3), que **no** es una lista:

```
suma = 0
for fila in temps:
    suma += fila[3]
print(suma / len(temps))
```

Recorrer una lista de listas

El promedio de 2016 (la fila 2):

```
print(sum(temps[2]) / len(temps[2]))
```

El promedio de abril (la columna 3), que **no** es una lista:

```
suma = 0
for fila in temps:
    suma += fila[3]
print(suma / len(temps))
```

Una fila se saca directo; una columna hay que recorrerla.

Construir una lista de listas

Las notas de cada alumno, leídas desde el teclado:

```
n = int(input('Cantidad de alumnos: '))
alumnos = []
for i in range(n):
    nombre = input('Nombre: ')
    nota = float(input('Nota: '))
    alumnos.append([nombre, nota])     # <- append de una lista
print(alumnos)
```

Construir una lista de listas

Las notas de cada alumno, leídas desde el teclado:

```
n = int(input('Cantidad de alumnos: '))
alumnos = []
for i in range(n):
    nombre = input('Nombre: ')
    nota = float(input('Nota: '))
    alumnos.append([nombre, nota])     # <- append de una lista
print(alumnos)
```

Ojo: `append([nombre, nota])` agrega **un** elemento (que es una lista). Con `append(nombre, nota)` da error.

Desempaquetado

Una lista se puede repartir en varias variables de una vez:

```
x, y = [10, 30]
print(x, y)         # 10 30
```

Desempaquetado

Una lista se puede repartir en varias variables de una vez:

```
x, y = [10, 30]
print(x, y)         # 10 30
```

Es muy cómodo al recorrer una lista de listas:

```
puntos = [[1, 2], [3, 4], [5, 6]]
for x, y in puntos:
    print('distancia:', (x**2 + y**2)**0.5)
```

Desempaquetado

Una lista se puede repartir en varias variables de una vez:

```
x, y = [10, 30]
print(x, y)         # 10 30
```

Es muy cómodo al recorrer una lista de listas:

```
puntos = [[1, 2], [3, 4], [5, 6]]
for x, y in puntos:
    print('distancia:', (x**2 + y**2)**0.5)
```

La cantidad de variables debe coincidir con el largo de la lista interior.

Ordenar listas de listas

`sort` compara **el primer elemento** de cada lista interior; si hay empate, pasa al segundo.

```
alumnos = [['Pedro', 50], ['Ana', 70], ['Luis', 30]]
alumnos.sort()
# [['Ana', 70], ['Luis', 30], ['Pedro', 50]]
```

Ordenar listas de listas

`sort` compara **el primer elemento** de cada lista interior; si hay empate, pasa al segundo.

```
alumnos = [['Pedro', 50], ['Ana', 70], ['Luis', 30]]
alumnos.sort()
# [['Ana', 70], ['Luis', 30], ['Pedro', 50]]
```

¿Y si queremos ordenar **por nota**? Se construye una lista nueva con la nota adelante:

```
por_nota = []
for nombre, nota in alumnos:
    por_nota.append([nota, nombre])
por_nota.sort()
```

Elegir al azar: choice

```
from random import choice

colores = ['azul', 'rojo', 'verde']
print(choice(colores))     # uno cualquiera de los tres
```

Elegir al azar: choice

```
from random import choice

colores = ['azul', 'rojo', 'verde']
print(choice(colores))     # uno cualquiera de los tres
```

- Retorna **un elemento** de la lista, elegido al azar.
- La lista no se modifica, y el elemento puede repetirse entre llamadas.

Elegir al azar: choice

```
from random import choice

colores = ['azul', 'rojo', 'verde']
print(choice(colores))     # uno cualquiera de los tres
```

- Retorna **un elemento** de la lista, elegido al azar.
- La lista no se modifica, y el elemento puede repetirse entre llamadas.

Compare con `randint`, que entrega un número de un rango.

Ruteo

Ruteo: ¿qué hace esta función?

```
def f(lista, x):
    c = 0
    for e in lista:
        if e == x:
            c += 1
    return c

print(f([3, 1, 3, 5, 3], 3))
```

Rutee la función.

e	c	salida

Ruteo: ¿qué hace esta función?

```
def f(lista, x):
    c = 0
    for e in lista:
        if e == x:
            c += 1
    return c

print(f([3, 1, 3, 5, 3], 3))
```

Rutee la función.

e	c	salida

Retorna cuántas veces aparece x: imprime 3.

Es lo que hace `lista.count(x)`.

Ruteo

```
a = [1, 2, 3]
b = a
c = a[:]
b.append(4)
c.append(5)
a[0] = 0
print(a)
print(b)
print(c)
```

¿Qué imprime?

Ruteo

```
a = [1, 2, 3]
b = a
c = a[:]
b.append(4)
c.append(5)
a[0] = 0
print(a)
print(b)
print(c)
```

¿Qué imprime?

```
[0, 2, 3, 4]
[0, 2, 3, 4]
[1, 2, 3, 5]
```

`b` es un alias de `a`; `c` es una copia.

Errores frecuentes

Encuentre los tres errores

Pedir números hasta que se ingrese `salir`, ordenarlos de mayor a menor y separar pares de impares.

```
ingreso = input('Ingrese un numero: ')
nros = list[]

while ingreso != 'salir':
    nros.append(int(ingreso))
    ingreso = input('Ingrese un numero: ')

nros = nros.sort()
nros.reverse()

pares = []
impares = []
for i in nros:
    if i % 2 == 0:
        pares.append(i)
    else:
        impares += i
```

Encuentre los tres errores

(1) `nros = list[]` no es una lista vacía. Se escribe `nros = []` (o `list()`).

Encuentre los tres errores

(1) `nros = list[]` no es una lista vacía. Se escribe `nros = []` (o `list()`).

(2) `nros = nros.sort()` deja `nros` en `None`. Debe ser sólo `nros.sort()`.

Encuentre los tres errores

(1) `nros = list[]` no es una lista vacía. Se escribe `nros = []` (o `list()`).

(2) `nros = nros.sort()` deja `nros` en `None`. Debe ser sólo `nros.sort()`.

(3) `impares += i` intenta sumar un entero a una lista. Debe ser `impares.append(i)`.

Encuentre los tres errores

(1) `nros = list[]` no es una lista vacía. Se escribe `nros = []` (o `list()`).

(2) `nros = nros.sort()` deja `nros` en `None`. Debe ser sólo `nros.sort()`.

(3) `impares += i` intenta sumar un entero a una lista. Debe ser `impares.append(i)`.

Note que `impares += [i]` **sí** funcionaría, porque ahí se concatenan dos listas.

Encuentre el error

Eliminar los negativos de una lista.

```
nros = [1, -2, -3, 4]
for n in nros:
    if n < 0:
        nros.remove(n)
print(nros)      # [1, -3, 4]
```

Encuentre el error

Eliminar los negativos de una lista.

```
nros = [1, -2, -3, 4]
for n in nros:
    if n < 0:
        nros.remove(n)
print(nros)      # [1, -3, 4]
```

Se modifica la lista mientras se recorre: al eliminar el `-2`, el `-3` corre a su posición y el `for` se lo salta.

```
positivos = []
for n in nros:
    if n >= 0:
        positivos.append(n)
```

Encuentre el error

Sumarle 5 a todas las notas.

```
notas = [55, 70, 48]
for n in notas:
    n = n + 5
print(notas)      # [55, 70, 48]
```

Encuentre el error

Sumarle 5 a todas las notas.

```
notas = [55, 70, 48]
for n in notas:
    n = n + 5
print(notas)      # [55, 70, 48]
```

`n` es una copia del valor: cambiarla no toca la lista. Hay que recorrer por índice.

```
for i in range(len(notas)):
    notas[i] = notas[i] + 5
```

Encuentre el error

Mostrar la posición de un elemento.

```
a = [1, 2, 3]
print(a.index(9))
```

Encuentre el error

Mostrar la posición de un elemento.

```
a = [1, 2, 3]
print(a.index(9))
```

`ValueError`: `index` exige que el elemento exista. Pregunte antes con `in`.

```
if 9 in a:
    print(a.index(9))
else:
    print('no está')
```

Lo que usamos en esta unidad

sí

- Creación de listas, incluidas las listas de listas.
- Acceso `[i]` y rebanadas `[a:b]`.
- Operadores `+`, `*`, `in`, `not in`, `del`.
- Funciones `len`, `sum`, `min`, `max`.
- Métodos `append`, `insert`, `remove`, `index`, `count`, `sort`, `reverse`.
- `for` y `range`; contraste con `while`.
- Mutabilidad, alias y copia.
- Doble índice y ciclo anidado.
- Desempaquetado; `choice` de `random`.

NO

- Tuplas: usamos listas de listas.
- Parámetros de `sort()` (`key`, `reverse`).
- `map()`.
- Modificar una lista dentro de una función.
- `break` y `continue`.

Para ordenar descendente: `sort()` y después `reverse()`, en dos instrucciones.

Ejercicios

Ejercicios de calentamiento

(a) Lea números hasta que se ingrese `salir`, ordénelos de mayor a menor e indique cuántos pares y cuántos impares hay.

(b) Dada una lista de palabras, encuentre la **más larga**.

(c) Escriba una función que, dada una lista de enteros, retorne una lista **nueva** que mantenga el orden original pero **sin repetidos**.

(d) Repita (c) suponiendo que la lista original está **ordenada**. ¿Cómo aprovecharía esa característica?

Ejercicios

(a) Dada una lista de números, retorne una lista donde los **adyacentes iguales** queden reducidos a un solo elemento.

```
[1, 2, 2, 3, 3, 3, 1]   ->   [1, 2, 3, 1]
```

(b) Dada una lista de strings, retorne la lista **ordenada**, pero agrupando al inicio todos los que empiezan con `'x'`.

```
['mix', 'xyz', 'apple', 'xanadu', 'aardvark']
->  ['xanadu', 'xyz', 'aardvark', 'apple', 'mix']
```

Ejercicio: desviación estándar

La desviación estándar de una lista de n números es:

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=0}^{n-1}(x_i - \mu)^2}, \qquad \mu = \frac{1}{n}\sum_{i=0}^{n-1} x_i$$

Escriba las funciones `promedio(lista)` y `desviacion(lista)`.

Ejercicio: desviación estándar

La desviación estándar de una lista de n números es:

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=0}^{n-1}(x_i - \mu)^2}, \qquad \mu = \frac{1}{n}\sum_{i=0}^{n-1} x_i$$

Escriba las funciones `promedio(lista)` y `desviacion(lista)`.

Sugerencia: `desviacion` debe llamar a `promedio`. Note que hay que recorrer la lista **dos veces**: una para el promedio y otra para la suma de los cuadrados.

Ejercicio: `merge`

Escriba la función `merge(lista1, lista2)` que recibe dos listas de números **ya ordenadas** ascendentemente, y retorna una lista nueva, también ordenada, con la mezcla de ambas.

```
merge([1, 4, 9], [2, 3, 10])   ->   [1, 2, 3, 4, 9, 10]
```

Ejercicio: `merge`

Escriba la función `merge(lista1, lista2)` que recibe dos listas de números **ya ordenadas** ascendentemente, y retorna una lista nueva, también ordenada, con la mezcla de ambas.

```
merge([1, 4, 9], [2, 3, 10])   ->   [1, 2, 3, 4, 9, 10]
```

La idea es aprovechar el orden de las listas de entrada: no vale concatenarlas y ordenar el resultado.

Piénselo como mezclar dos mazos de cartas que ya están ordenados: se van comparando las dos cartas de arriba.

Ejercicio: puntos

Una lista de puntos del plano viene como una lista de listas:

```
puntos = [[1, 2], [3, 4], [-5, 6], [0, 1]]
```

(a) Escriba una función que retorne el punto **más lejano del origen**.

(b) Desafío: escriba una función que retorne la lista ordenada desde el punto **más cercano** al más lejano del origen.

Ejercicio: temperaturas

```
años = [2018, 2017, 2016, 2015]
temps = [
    [25, 25, 22, 19, 16, 12, 11, 13, 15, 18, 23, 23],
    [27, 25, 22, 18, 16, 11, 12, 12, 14, 17, 19, 28],
    [24, 25, 22, 16, 13, 12, 11, 12, 13, 18, 22, 23],
    [26, 24, 24, 20, 15, 13, 12, 13, 15, 16, 19, 24]
]
```

Escriba funciones que respondan:

(a) la temperatura de un año y un mes dados **(b)** la temperatura promedio de 2016

(c) la temperatura promedio de abril **(d)** cuál ha sido el octubre más frío

Ejercicio: notas

```
alumnos = [
    ['Alberto Gonzalez', 40, 30, 70],
    ['Francisca Almonacid', 100, 40],
    ['Pedro Reyes', 30, 50],
    ['Juan Campos', 30, 60, 30, 70],
    ['Andrea Zamora', 30]
]
```

(a) Calcule e imprima el promedio de cada alumno. Note que **no todos tienen la misma cantidad de notas**.

(b) Identifique a los alumnos con promedio menor a 55. Si tienen **menos de 3 notas**, pida una nota más por teclado y agréguela al final de su lista.

Ejercicio: cine Pynemark

El reporte de asistencia llega como una lista de listas: cada **fila** es una sala y cada **columna** una película.

```
cine = [
    [34, 65, 87, 34, 45],     # sala 1
    [56, 78, 76, 23, 56],     # sala 2
    [45, 67, 23, 45, 34],     # sala 3
    [45, 67, 56, 78, 54]      # sala 4
]
```

(a) `sala_mas_usada(cine)`: retorna el número de la sala que tuvo más personas en total.

(b) `pelicula_mas_famosa(cine)`: retorna el número de la película con más espectadores en todo el cine.

Ejercicio: cine Pynemark

El reporte de asistencia llega como una lista de listas: cada **fila** es una sala y cada **columna** una película.

```
cine = [
    [34, 65, 87, 34, 45],     # sala 1
    [56, 78, 76, 23, 56],     # sala 2
    [45, 67, 23, 45, 34],     # sala 3
    [45, 67, 56, 78, 54]      # sala 4
]
```

(a) `sala_mas_usada(cine)`: retorna el número de la sala que tuvo más personas en total.

(b) `pelicula_mas_famosa(cine)`: retorna el número de la película con más espectadores en todo el cine.

(a) suma filas; (b) suma columnas.

Ejercicio: turnos de enfermería

El personal y sus turnos se guardan en dos listas de listas:

```
profesionales = [
    [125, 'Ann',  'M'],
    [271, 'Beth', 'E'],
    [443, 'John', 'G'],
    [105, 'Ben',  'M'],
    [85,  'Fer',  'E']
]
```

```
turnos = [
    [443, [5, 8, 2]],
    [126, [6, 2, 4, 7]],
    [271, [1, 9]],
    [85,  [2, 9]]
]
```

El tipo es Maternidad (`'M'`), Emergencias (`'E'`) o General (`'G'`). Cada turno asocia un identificador con la lista de bloques (del 1 al 10) en que esa persona trabaja.

Ejercicio: turnos de enfermería

(a) `turno_emergencias(profesionales, turnos)`: retorna una lista con todos los bloques, **sin repetición**, en que trabaja personal de Emergencias.

```
>>> turno_emergencias(profesionales, turnos)
[1, 9, 2]
```

(b) `buscar_profesional(profesionales, identificador)`: retorna el nombre de esa persona, o el string vacío si el identificador no está.

```
>>> buscar_profesional(profesionales, 443)
'John'
```

(c) `jefe(profesionales, turnos)`: retorna el nombre de quien tiene **menos** turnos asignados. Si hay empate, cualquiera de ellos.

```
>>> jefe(profesionales, turnos)
'Beth'
```

Ejercicio: tienda de notebooks

```
# [modelo, marca, pantalla, RAM, tipo disco, capacidad, procesador, video]
productos = [
    ['8475HD', 'HP', 15.6, '8GB', 'DD', '1T', 'Intel Core i5', 'Nvidia GTX1050'],
    ['2175HD', 'lenovo', 14, '4GB', 'SSD', '512GB', 'Intel Core i5', 'Nvidia GTX1050'],
    ['JjfFHD', 'Asus', 14, '16GB', 'SSD', '256GB', 'Intel Core i7', 'Nvidia RTX2080Ti'],
    ...
]
# [modelo, precio, stock]
stock = [['8475HD', 387990, 10], ['2175HD', 327990, 4], ['JjfFHD', 424990, 1], ...]
```

(a) `stock_marca(marca, productos, stock)`: total de unidades disponibles de esa marca, en todos sus modelos. `0` si no hay ninguno. No distinga mayúsculas de minúsculas.

```
>>> stock_marca('hp', productos, stock)
31
```

Ejercicio: tienda de notebooks

(b) `filtrar_equipos(caracteristicas, productos, stock)`: recibe una lista con **tres** características (tamaño de pantalla, RAM y capacidad del disco) y retorna la lista de los equipos que las cumplen **todas**, incluyendo precio y stock, **ordenados por marca**. Lista vacía si no hay ninguno.

```
>>> filtrar_equipos([14, '16GB', '256GB'], productos, stock)
[['Asus', 'JjfFHD', 'SSD', 'Intel Core i7', 'Nvidia RTX2080Ti', 424990, 1]]

>>> filtrar_equipos([15.6, '16GB', '512GB'], productos, stock)
[]
```

Sugerencia: construya cada elemento del resultado con la marca adelante, y después ordene con `sort()`.

Desafío: cartón de bingo

Genere un cartón de bingo de 5 columnas, al azar. Los números van del 1 al 75, pero **cada columna tiene su rango**: la primera sólo del 1 al 15, la segunda del 16 al 30, y así sucesivamente.

El cartón debe quedar en una lista de listas, y no puede haber números repetidos.

Desafío: cartón de bingo

Genere un cartón de bingo de 5 columnas, al azar. Los números van del 1 al 75, pero **cada columna tiene su rango**: la primera sólo del 1 al 15, la segunda del 16 al 30, y así sucesivamente.

El cartón debe quedar en una lista de listas, y no puede haber números repetidos.

Sugerencia: para cada columna, construya primero la lista de sus 15 candidatos con `range`, y después vaya sacando elementos al azar con `choice` y `remove`.

Desafío: criba de Eratóstenes

Para encontrar todos los primos hasta `n`:

1. Cree una lista de `n+1` casilleros, todos en `True`.
2. Marque como `False` las posiciones `0` y `1`.
3. Recorra desde `2`: cada vez que encuentre una posición en `True`, marque como `False` **todos sus múltiplos**.
4. Al final, las posiciones que siguen en `True` son los primos.

Desafío: criba de Eratóstenes

Para encontrar todos los primos hasta `n`:

1. Cree una lista de `n+1` casilleros, todos en `True`.
2. Marque como `False` las posiciones `0` y `1`.
3. Recorra desde `2`: cada vez que encuentre una posición en `True`, marque como `False` **todos sus múltiplos**.
4. Al final, las posiciones que siguen en `True` son los primos.

Note la idea: el **índice** es el número, y el **valor** dice si sigue siendo candidato.